I2236 INTRODUCTION TO EMBEDDED SYSTEMS

LECTURE 4 – PART 1

INTERFACING PERIPHERALS

Bilal Daass
04/02/2026

COURSE OVERVIEW

Module 1: Introduction & Foundations (Week 1–2)

Module 2: Arduino Ecosystem & Programming Basics (Week 3–4)

Module 3: Digital & Analog Interfacing (Week 5–6)

Module 4: Interfacing Peripherals (Week 7–8)

Module 5: Sensors & Actuators (Week 9–10)

Module 6: Communication & Integration (Week 11–12)

Module 7: Project (Week 13–15)

- **Input devices:**
  - **Keypad,**
  - **Ultrasonic Distance Sensor,**
  - **Passive infrared sensor (PIR)**
  - **Other sensors (Humidity and temperature sensor, Hall effect sensor, Accelerometer and gyroscope)**
- **Output devices:**
  - **LCD,**
  - **7-segment display.**
  - **Servo motor**
  - **IR Remote**

KEYPAD

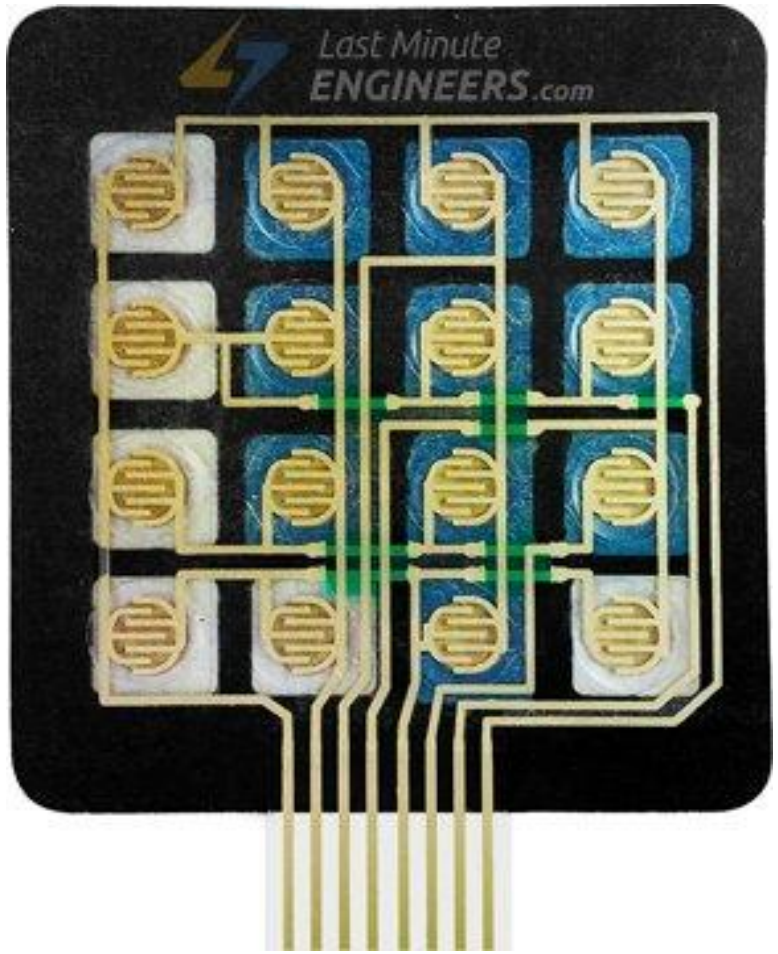
KEYPAD

- Matrix keypads are the kind of keypads you see on cell phones, calculators, microwaves ovens, door locks, etc. In electronics, they are a great way to let users interact with your project and are often needed to navigate menus, punch in passwords and control robots.

- Membrane keypads are made of a thin, flexible membrane material. They do come in may sizes 4×3, 4×4, 4×1 etc. Regardless of their size, they all work in the same way.

- One of the great thing about them is that they come with an adhesive backing so you can attach it easily.

KEYPAD

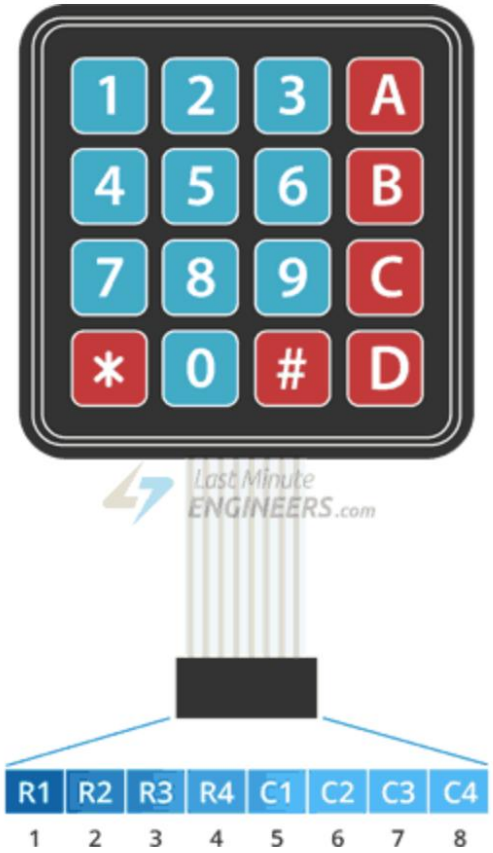


- 4×4 keypad has total 16 keys. Beneath each key is a special membrane switch.
- All these membrane switches are connected to each other with conductive trace underneath the pad forming a matrix of 4×4 grid
- If you had used 16 individual push buttons, you would have required 17 input pins (one for each key and a ground pin) in order to make them work.
- However, with matrix arrangement, you only need 8 microcontroller pins (4-columns and 4-rows) to scan through the pad.

KEYPAD

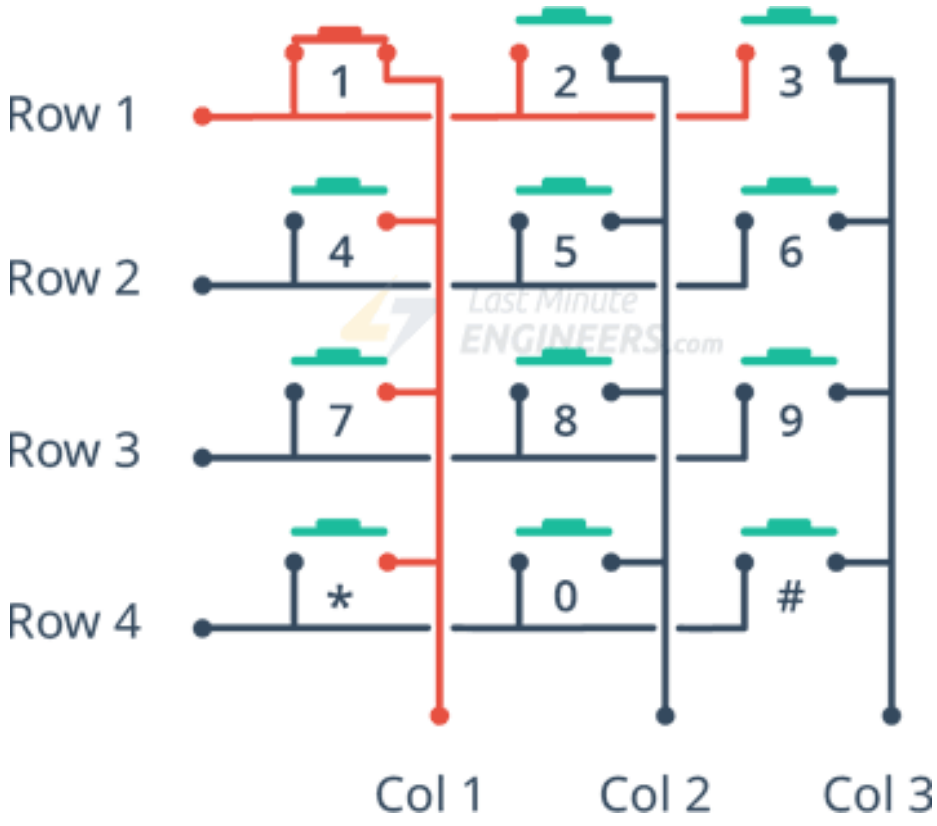
4×3 keypad

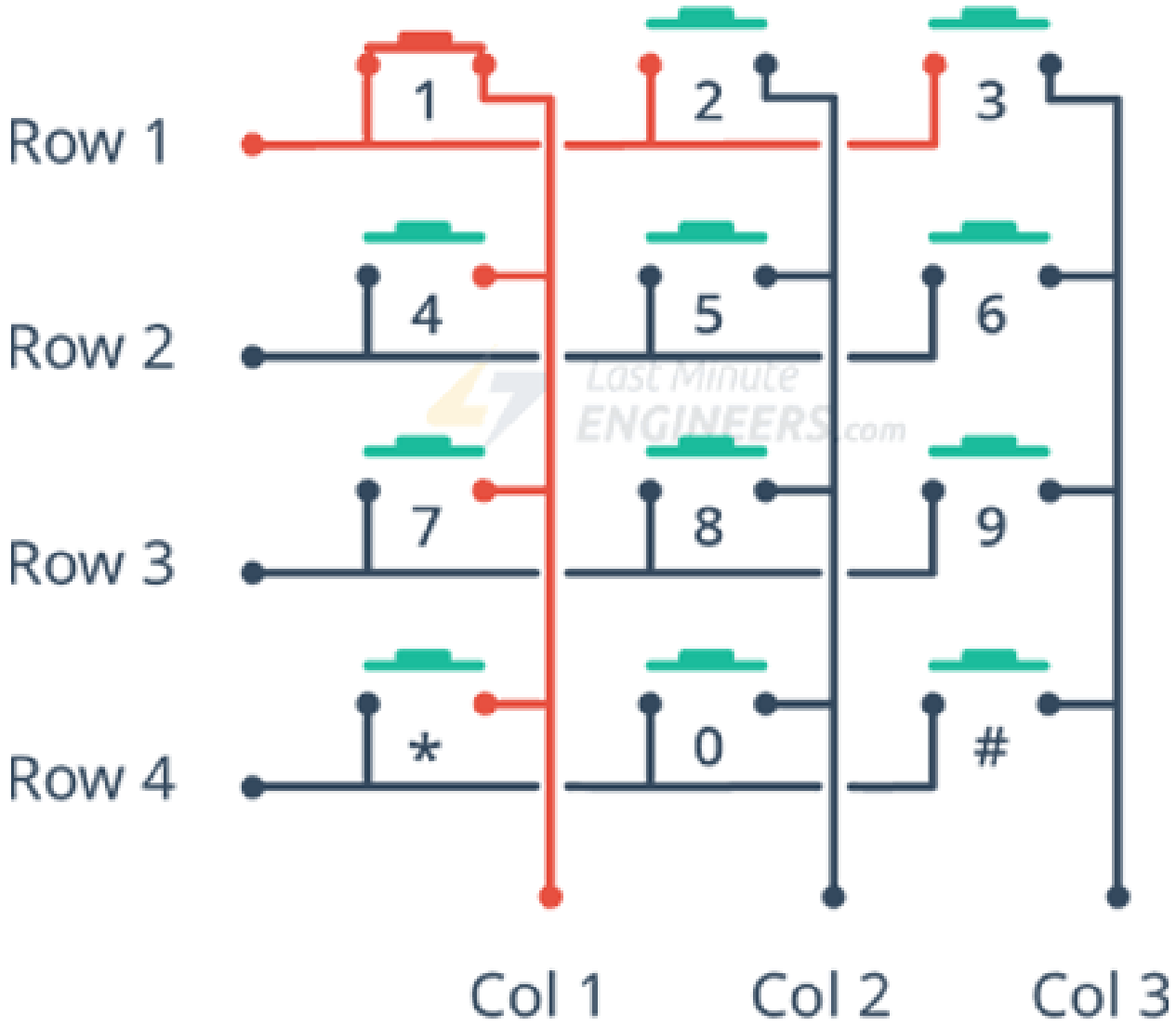
4×4 keypad



KEYPAD-THE WORKING PRINCIPLE

- Pressing a button shorts one of the row lines to one of the column lines, allowing current to flow between them.
- For example, when key ‘4’ is pressed, column 1 and row 2 are shorted.
- A microcontroller can scan these lines for a button-pressed state. To do this, it follows below procedure.
  1. Microcontroller sets all the column and row lines to input.
  2. Then, it picks a row and sets it HIGH.
  3. After that, it checks the column lines one at a time.
  4. If the column connection stays LOW, the button on the row has not been pressed.
  5. If it goes HIGH, the microcontroller knows which row was set HIGH, and which column was detected HIGH when checked.
  6. Finally, it knows which button was pressed that corresponds to detected row & column.

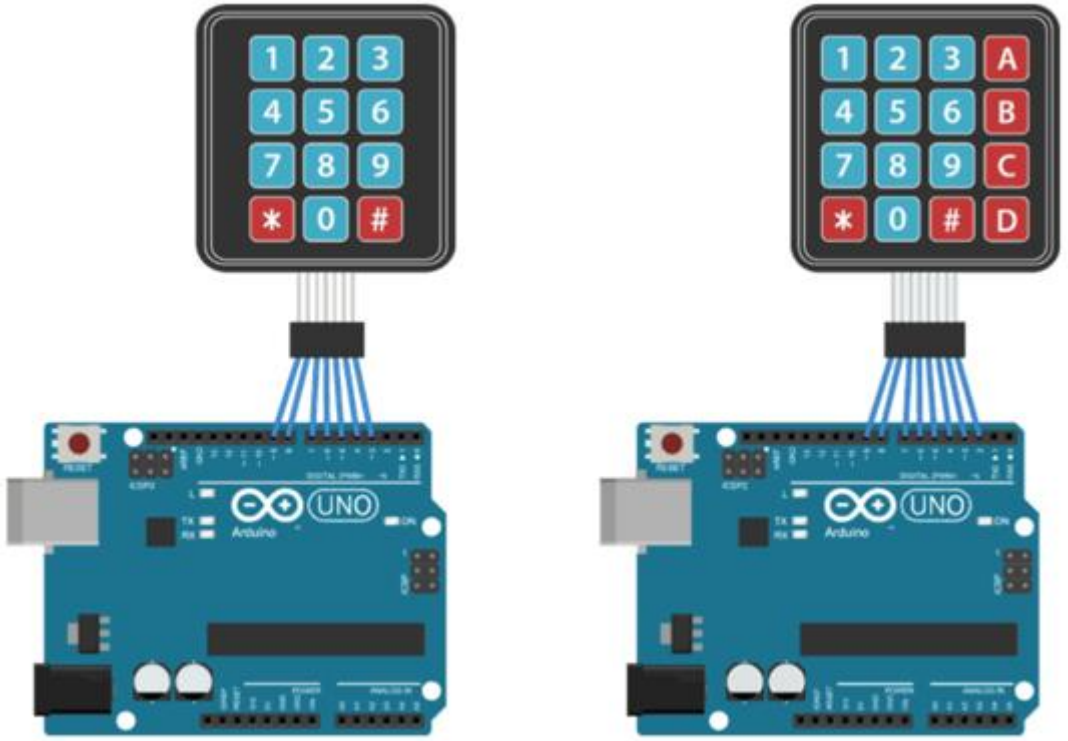



Row 1
Row 2
Row 3
Row 4
1
2
3
4
5
6
7
8
9
*
0
#
Col 1
Col 2
Col 3


1 2 3 A
4 5 6 B
7 8 9 C
* 0 # D

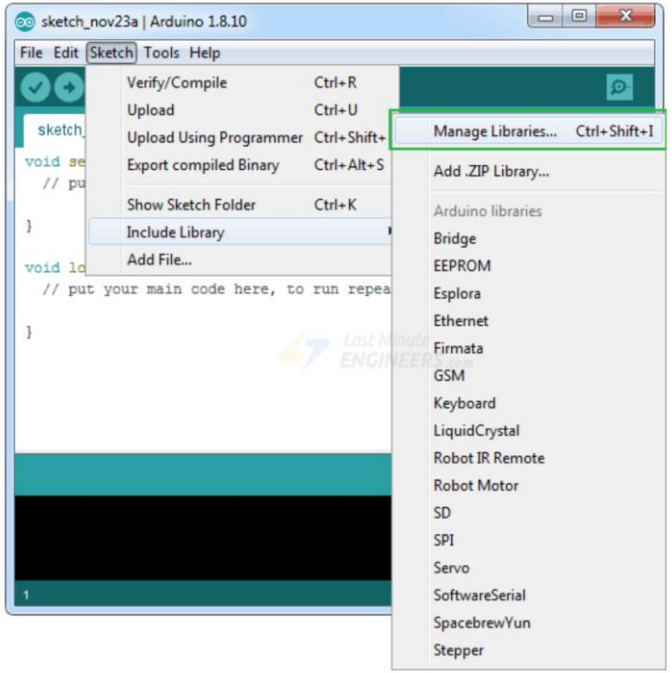
KEYPAD - WIRING

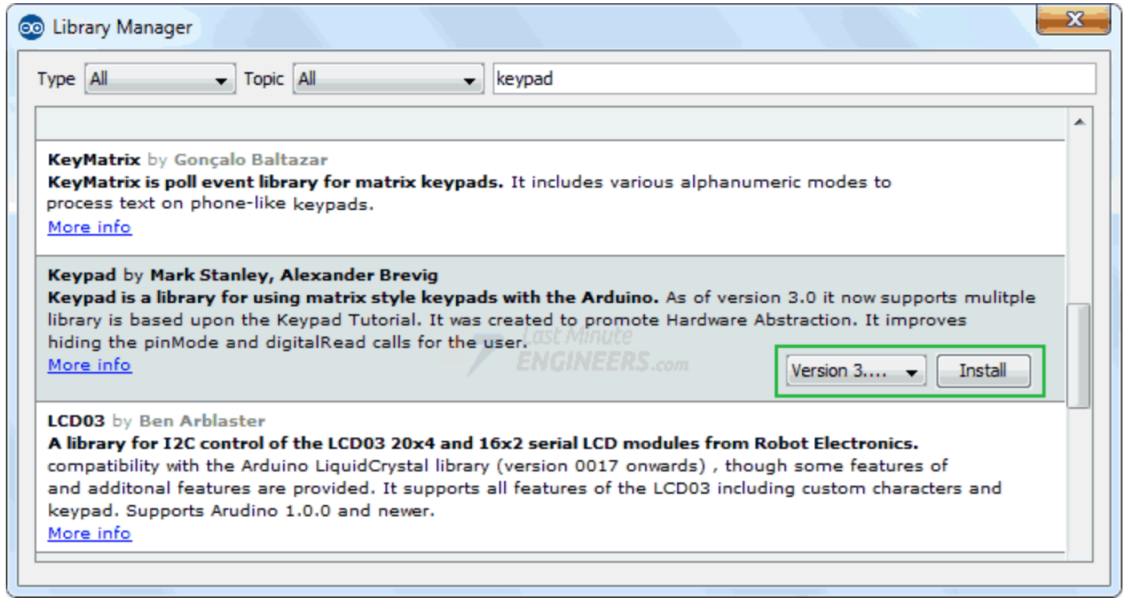
- The connections are pretty straightforward. Start by connecting pin 1 of keypad to digital pin 9 on Arduino. Now keep on connecting the pins leftwards like 2 with 8, 3 with 7 etc.



KEYPAD – INSTALLING LIBRARY

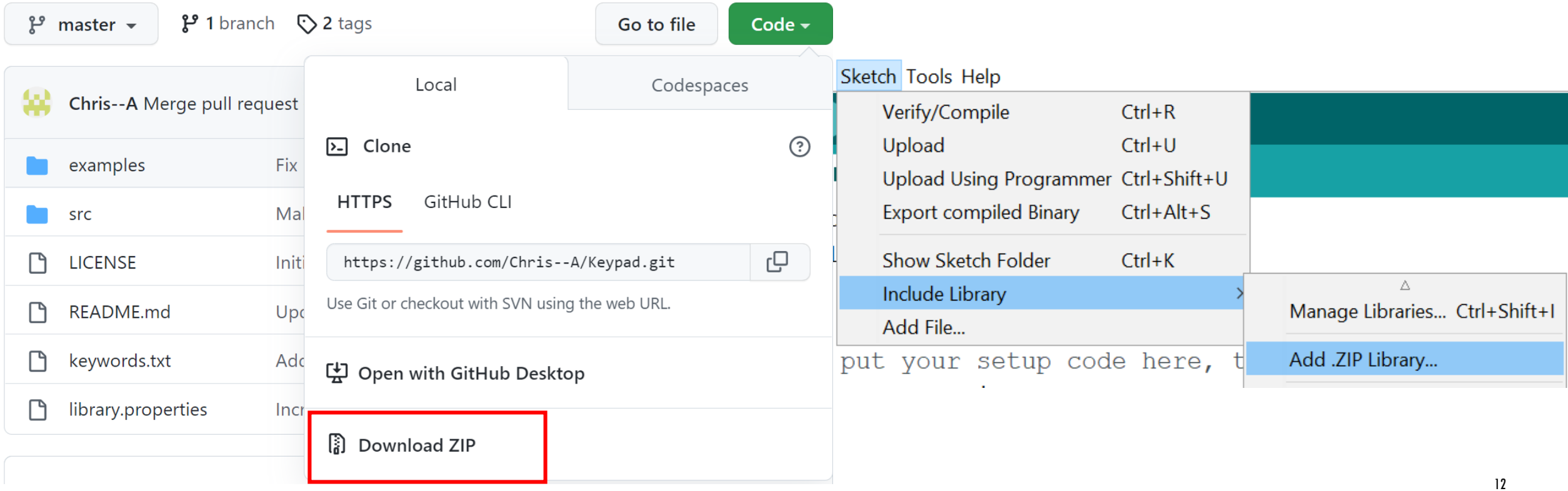
- In order to determine which key was pressed, we need to continuously scan rows & columns. Fortunately, Keypad.h was written to hide away this unnecessary complexity so that we can issue simple commands to know which key was pressed.
- To install the library navigate to the Sketch > Include Library > Manage Libraries…Wait for Library Manager to download libraries index and update list of installed libraries.





KEYPAD – INSTALLING LIBRARY

- Another method is to download the library from https://github.com/Chris--A/Keypad
- Then go to Sketch -> Include Library -> Add .ZIP Library and select the downloaded file



KEYPAD – CODE EXAMPLE

```
#include <Keypad.h>
const byte ROWS = 4;                    //four rows
const byte COLS = 4;                    //four columns
char hexaKeys[ROWS][COLS] = {    //define the symbols of the keypads
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','D','#','D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6};
byte colPins[COLS] = {5, 4, 3, 2};
Keypad customKeypad = Keypad( makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);
```

```
void setup(){
  Serial.begin(9600);
}
void loop(){
  char customKey = customKeypad.getKey();
  if (customKey){
    Serial.println(customKey);
  }
}
```

OUTPUT :

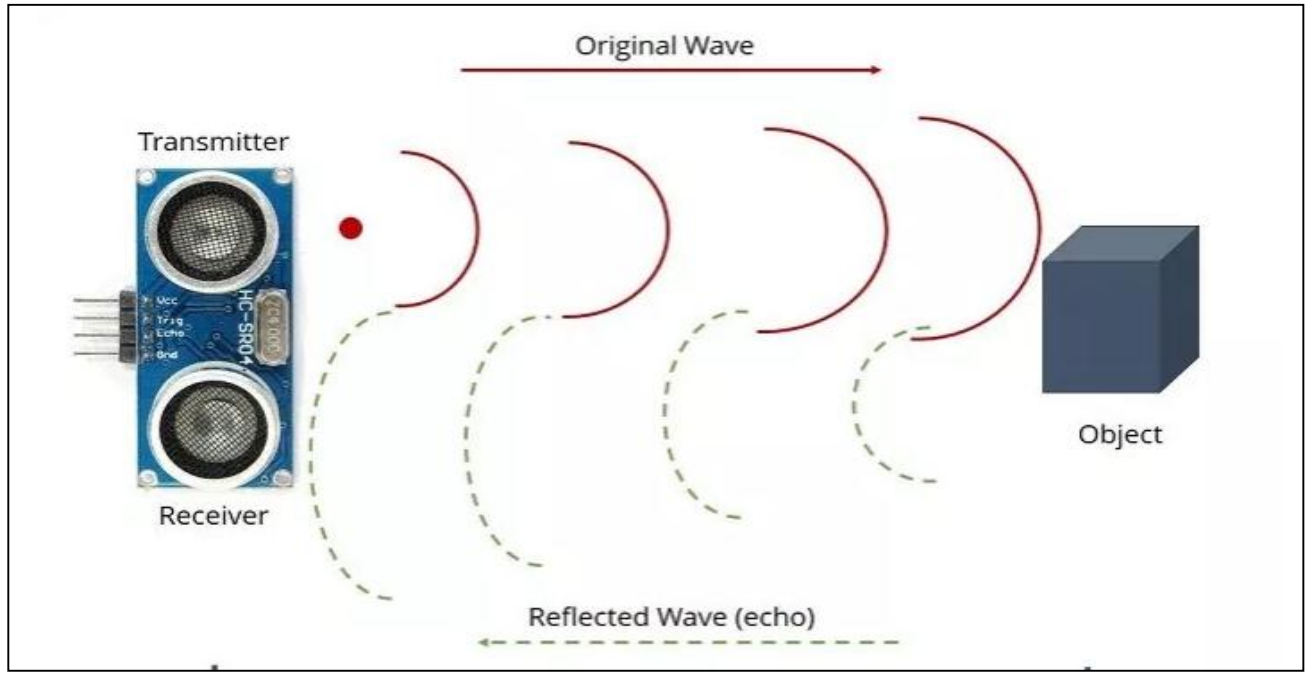
KEYPAD – USEFUL FUNCTIONS

- char **waitForKey()** Waits forever until someone presses a key. Warning – It blocks all other code until a key is pressed.
- **KeyState getState()** Returns the current state of any of the keys. The four states are IDLE, PRESSED, RELEASED and HOLD.
- boolean **keyStateChanged()** Let's you know when the key has changed from one state to another. For example, instead of just testing for a valid key you can test for when a key was pressed.
- **setHoldTime(unsigned int time)** Sets the amount of milliseconds the user will have to hold a button until the HOLD state is triggered.
- **setDebounceTime(unsigned int time**) Sets the amount of milliseconds the keypad will wait until it accepts a new keypress/keyEvent.
- **addEventListener(keypadEvent)** Triggers an event if the keypad is used.

ULTRASONIC DISTANCE SENSOR

ULTRASONIC DISTANCE SENSOR

The HC-SR04 ultrasonic distance sensor estimates distance by transmitting an ultrasonic sound wave and measuring the time taken to receive the echo.



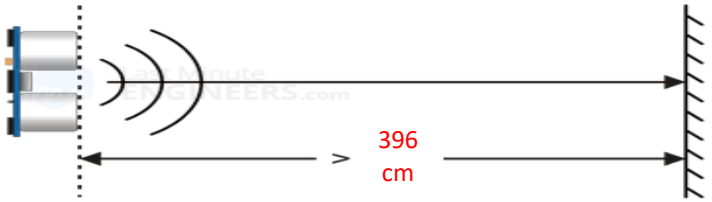
Distance between sensor and object:

$$d = \Delta t \times v \times \frac{1}{2}$$

- **v is the speed of sound in dry air at 20 °C:**
  v = 343 m / s
- **Δt is the time travel duration (Echo time - measured in microseconds (10^-6))**

ULTRASONIC DISTANCE SENSOR

- The frequency of the sound wave is 40kHz, which is above the upper limit of human hearing of 20kHz.
- The minimum and maximum measurable distances are 2cm and 4m, respectively.
- The accuracy is of the order of 3mm.
- For reliable distance measurements, the ultrasonic distance sensor should be perpendicular to the scanned surface.
- The sensor is small, easy to use in any robotics project since it operates on 5 volts, it can be hooked directly to an Arduino or any other 5V logic microcontrollers.

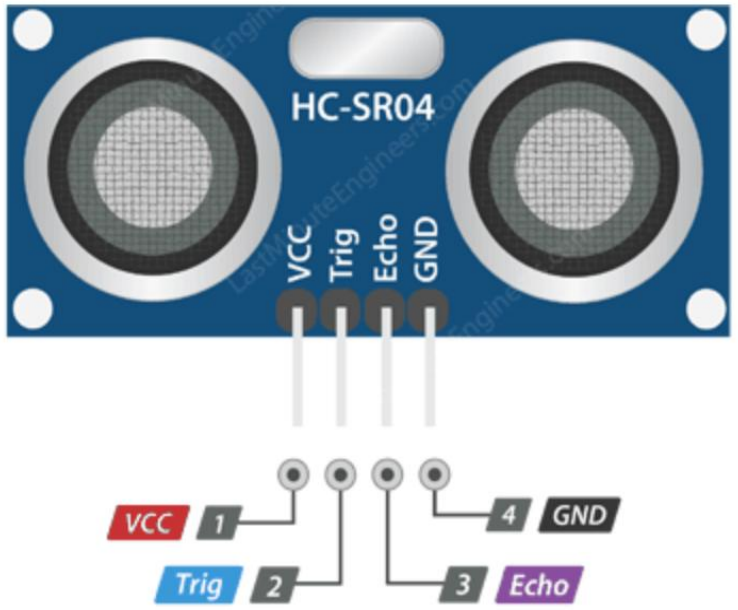


Application example

ULTRASONIC DISTANCE SENSOR

CONNECTIONS

1. VCC is the power supply.
2. Trig (Trigger) pin is used to trigger the ultrasonic sound pulses.
3. Echo pin produces a pulse when the reflected signal is received.
4. GND should be connected to the ground.



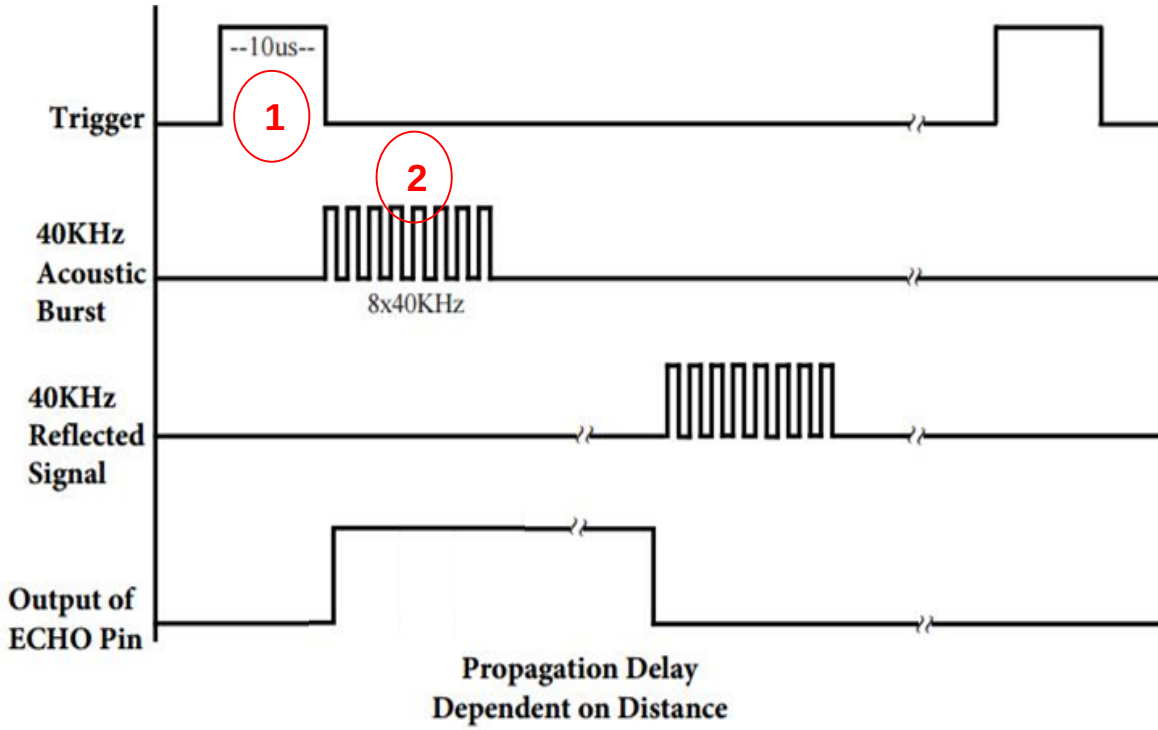
ULTRASONIC DISTANCE SENSOR

HOW DOES IT WORK?

1. First we set a pulse in the trigPin of duration at least 10 µS (10 microseconds).

```
digitalWrite(trigPin, HIGH); // set the trigger pin HIGH for 10µs
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
```

2. When it ends, the module sends 8 pulses at frequency 40 KHz. This 8-pulse pattern makes the “ultrasonic signature” from the device unique, allowing the receiver to differentiate the transmitted pattern from the ambient ultrasonic noise. The eight ultrasonic pulses travel through the air away from the transmitter.



ULTRASONIC DISTANCE SENSOR

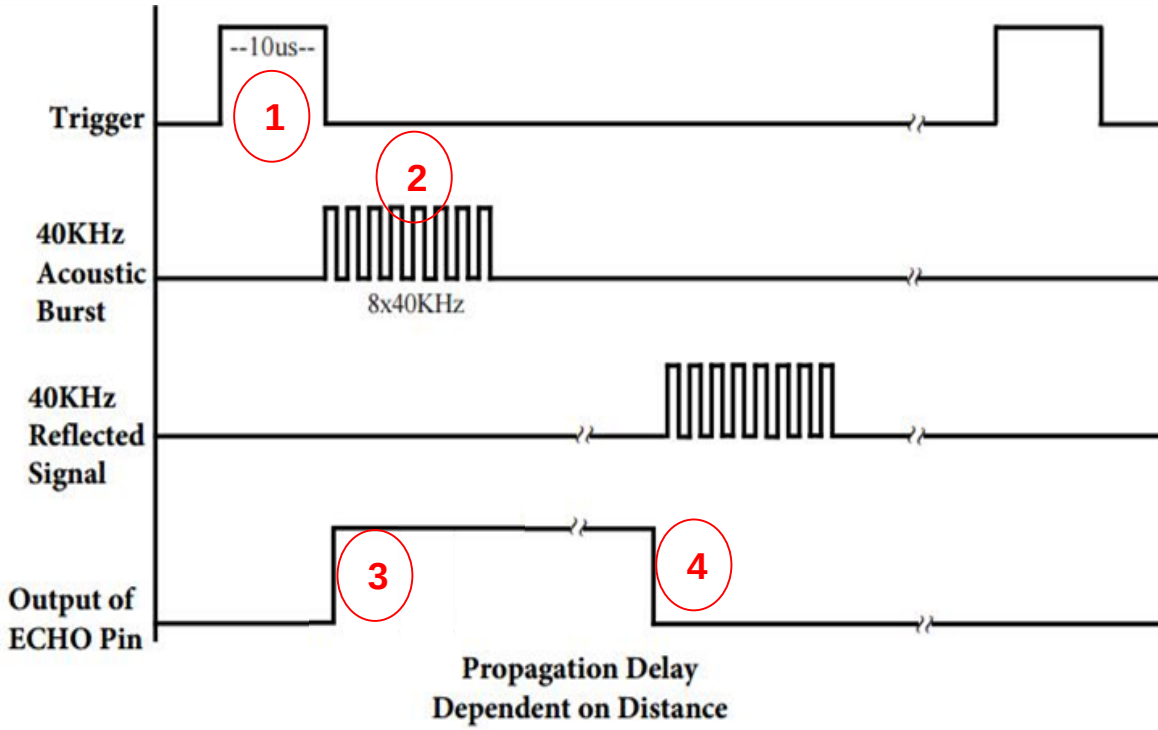
HOW DOES IT WORK?

3. At exactly the moment the first burst is sent, the echo pin is set to HIGH : pulseIn starts timing

```
duration = pulseIn(echoPin, HIGH);
// measure the echo time (µs)
```

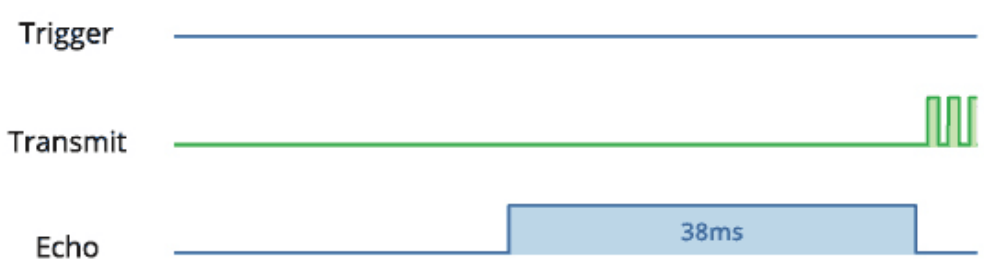
4. When the first pulse of the 40 kHz bounces in the object and gets back to the receiver, the echoPin becomes LOW. The pulseIn function stops the timer and returns the elapsed time.

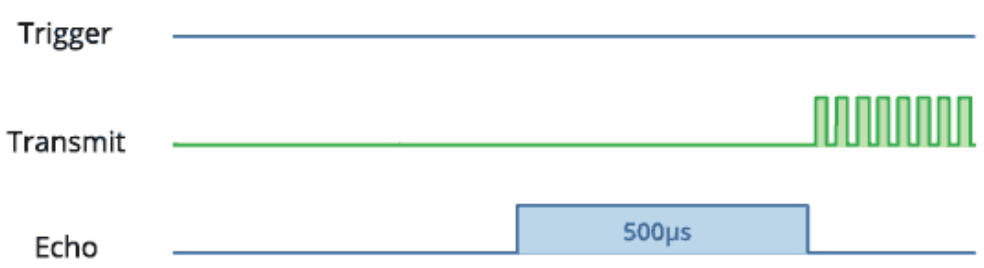
In case the transmitted pulses are not reflected back, the Echo signal will timeout after 38 ms and return low.



ULTRASONIC DISTANCE SENSOR

HOW DOES IT WORK?



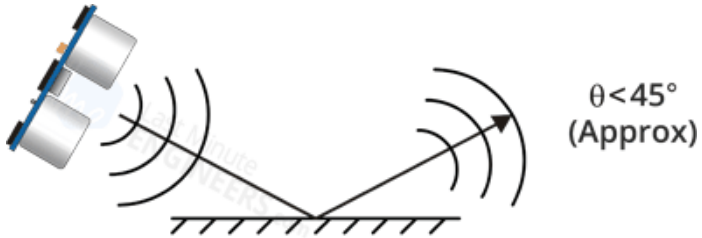


ULTRASONIC DISTANCE SENSOR

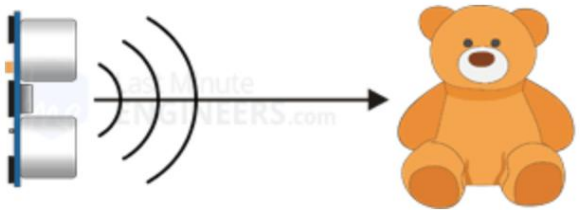
LIMITATIONS

- The following diagrams shows a few situations that the HC-SR04 is not designed to measure:

The object has its reflective surface at a shallow angle so that sound will not be reflected back towards the sensor.



Some objects with soft, irregular surfaces absorb rather than reflect sound

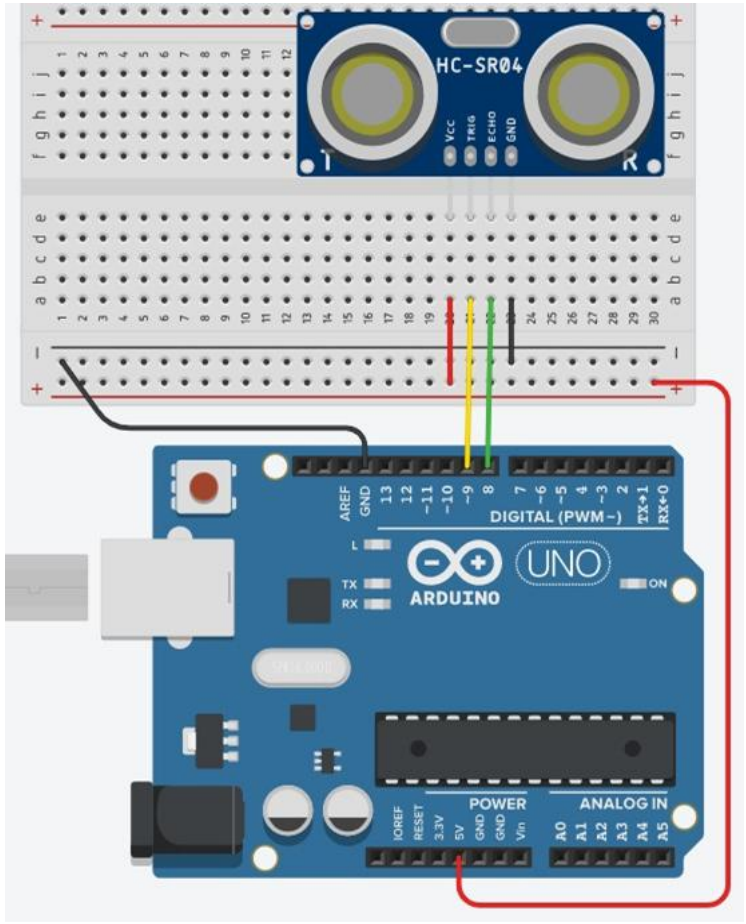


The object is too small to reflect enough sound back to the sensor.

ULTRASONIC DISTANCE SENSOR

CODE EXAMPLE

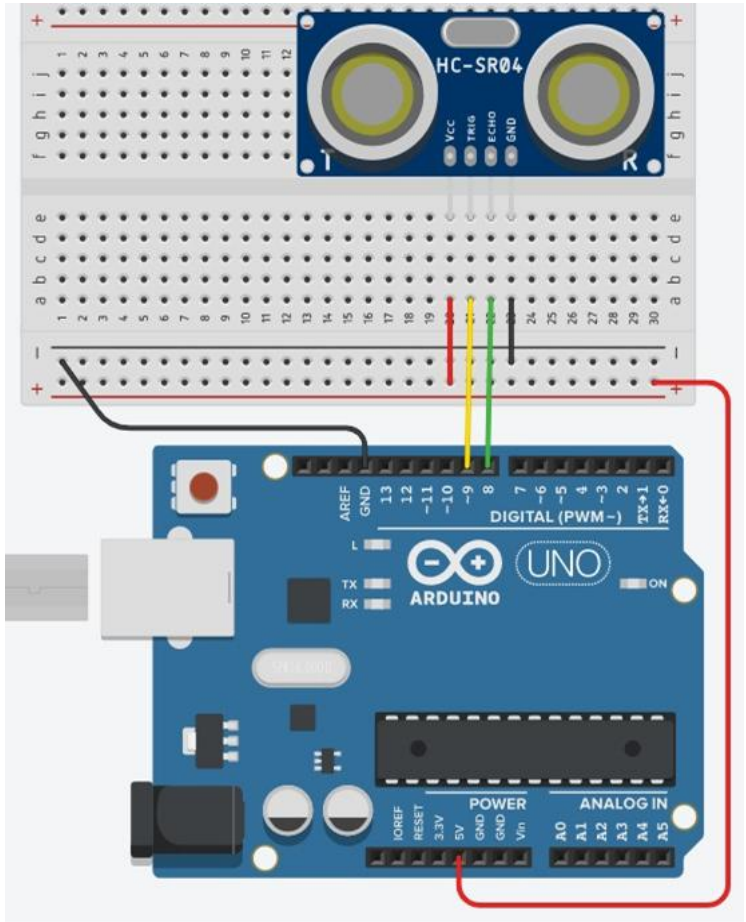
```
int trigPin = 9; // HC-SR04 trigger pin
int echoPin = 8; // HC-SR04 echo pin
float duration, distance;

void setup()
{
        pinMode(trigPin, OUTPUT); // Sets the trigPin as an OUTPUT
        pinMode(echoPin, INPUT); // Sets the echoPin as an INPUT

        Serial.begin(9600);
        // print some text in Serial Monitor
        Serial.println("Ultrasonic Sensor HC-SR04 Test");
        Serial.println("with Arduino UNO R3");
}
```



ULTRASONIC DISTANCE SENSOR

CODE EXAMPLE

```
void loop()
{
        digitalWrite(trigPin, LOW); // Clears the trigPin condition
        delayMicroseconds(2);
        digitalWrite(trigPin, HIGH); // set the trigger pin HIGH for 10µs
        delayMicroseconds(10);
        digitalWrite(trigPin, LOW);
        duration = pulseIn(echoPin, HIGH); // measure the echo time (µs)
        distance = (duration/2.0)*0.0343;// convert echo time to distance (cm)
        if(distance>=400 || distance<2)
              {
                        Serial.println("Out of range");
              }
        else
                {
                        Serial.print("Distance : ");
                        Serial.print(distance, 1);
                        Serial.println(" cm");
                }
        delay(1000);
}
```



- Passive infrared sensor (PIR)
- Humidity and temperature sensor
- Hall effect sensor
- Accelerometer and gyroscope

PASSIVE INFRARED SENSOR (PIR)

- The HR-SC501 passive infrared (PIR) sensor converts the infrared radiation into an output voltage
- The infrared radiation is due to body heat
- The sensor itself does not emit any energy but passively receives it.
- The white dome is a lens that expands the IR detector's field of vision.

- Applications:
  - Automatically turn light on when someone enters a room
  - Security alarm systems
  - Automatic door openers
  - Human detection by a robot

PASSIVE INFRARED SENSOR (PIR)

How PIR Motion Sensor Works?

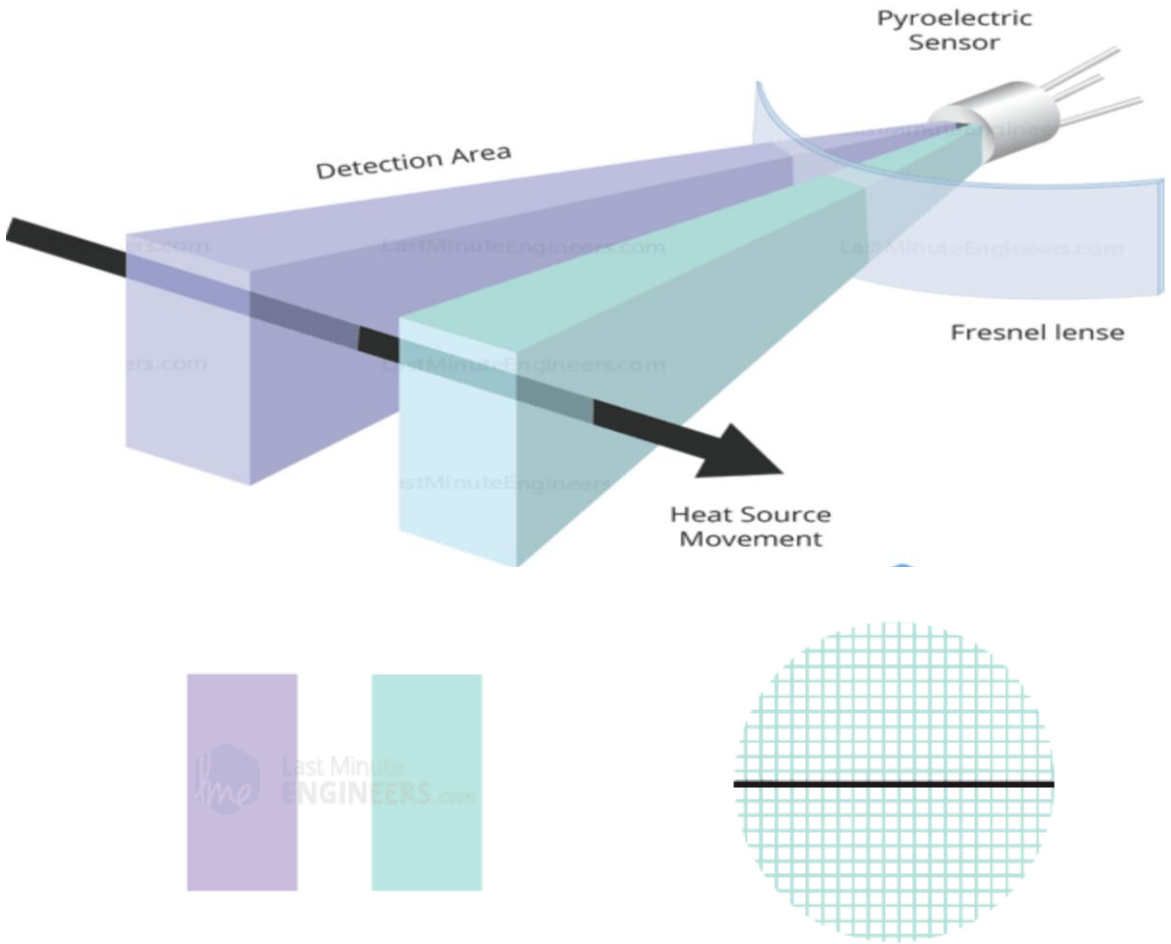
- All objects with a temperature above Absolute Zero (0 Kelvin / -273.15 °C) emit heat energy in the form of infrared radiation, including human bodies.
- The hotter an object is, the more radiation it emits.

- PIR sensor is specially designed to detect such levels of infrared radiation.
- The sensor reports a LOW signal by default.
- It reads the amount of ambient infrared light coming in.
- It triggers a HIGH signal for a certain period of time when the light levels change, indicating movement.
- It can tell you whether or not there is movement in a scene, but cannot detect distance.

PASSIVE INFRARED SENSOR (PIR)

How PIR Motion Sensor Works?

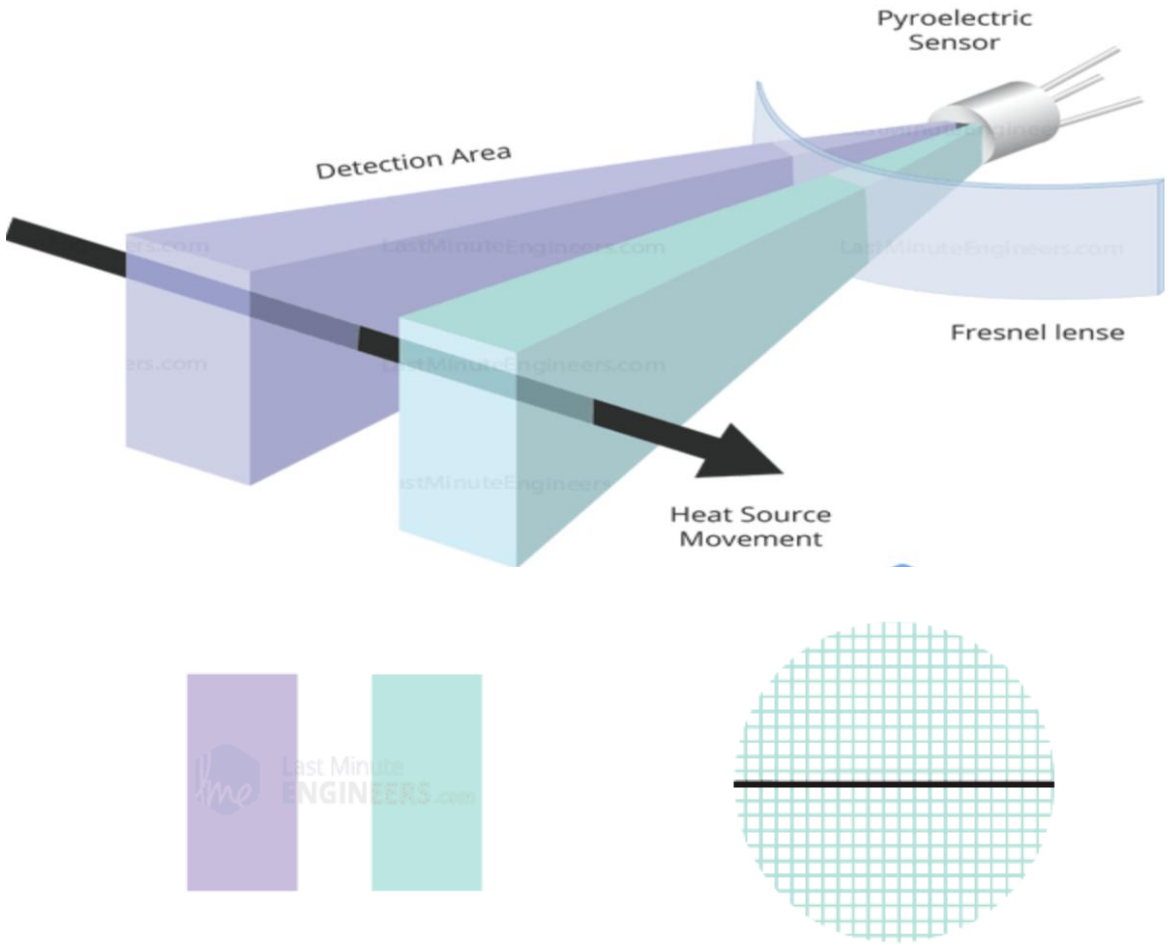
- It basically consists of two main parts: A Pyroelectric Sensor and A special lens called Fresnel lens which focuses the infrared signals onto the pyroelectric sensor.
- A Pyroelectric Sensor actually has two rectangular slots in it made of a material that allows the infrared radiation to pass.
- Behind these, are two separate infrared sensor electrodes, one responsible for producing a positive output and the other a negative output.
- The reason for that is that we are looking for a change in IR levels and not ambient IR levels.
- The two electrodes are wired up so that they cancel each other out.
- If one half sees more or less IR radiation than the other, the output will swing high or low.



PASSIVE INFRARED SENSOR (PIR)

How PIR Motion Sensor Works?

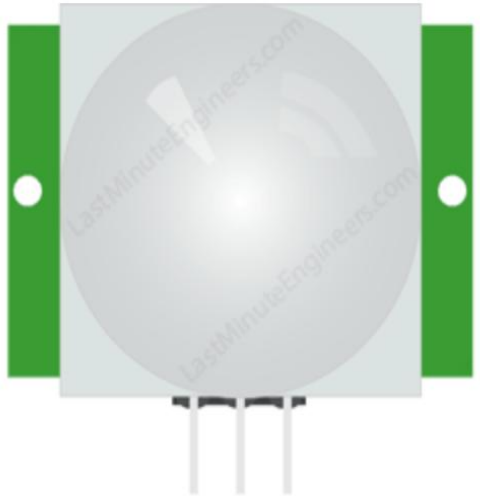
- When the sensor is idle, i.e. there is no movement around the sensor; both slots detect the same amount of infrared radiation, resulting in a zero output signal.
- But when a warm body like a human or animal passes by; it first intercepts one half of the PIR sensor, which causes a positive differential change between the two halves.
- When the warm body leaves the sensing area, the reverse happens, whereby the sensor generates a negative differential change. The corresponding pulse of signals results in the sensor setting its output pin high.



PASSIVE INFRARED SENSOR (PIR)

HC-SR501 PIR Sensor Wiring

The HC-SR501 has a 3-pin connector:



1. VCC should be connected to the 5V pin on the Arduino (The sensor has a built-in voltage regulator so it can be powered by any DC voltage from 4.5 to 12 volts, typically 5V is used)

2. Output pin (LOW indicates no motion is detected, HIGH means some motion has been detected).

3. GND should be connected to the ground of Arduino.

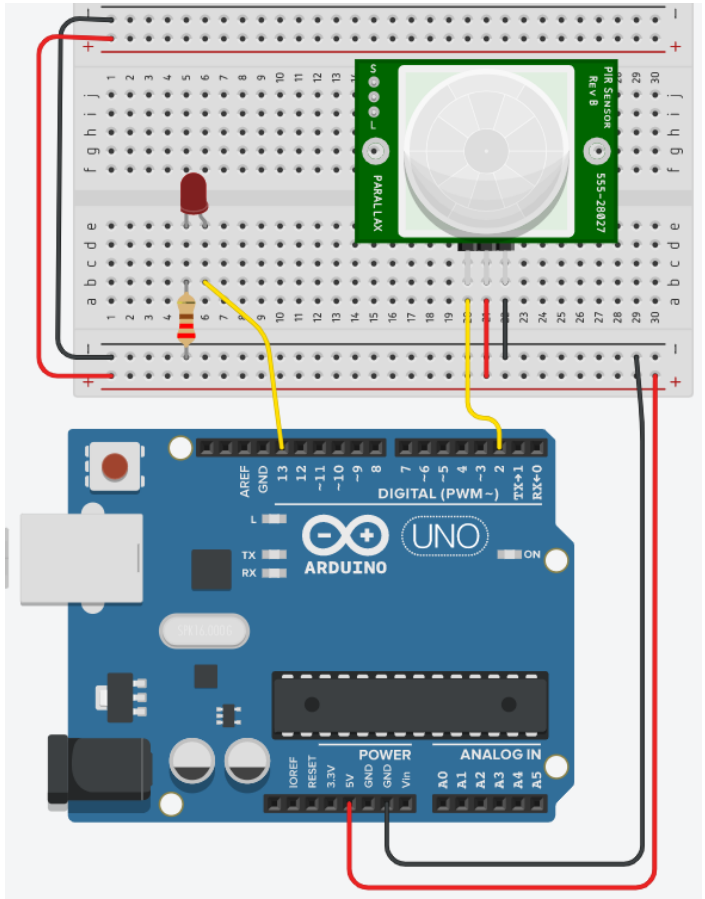
Some PIR motion sensors come with two adjustable potentiometers for changing the sensitivity and the duration of the activation signal.

- Sensitivity– This sets the maximum distance that motion can be detected. It ranges from 3 meters to approximately 7 meters.
- Time (duration of the activation) – This sets how long that the output will remain HIGH after detection. At minimum it is 3 seconds, at maximum it is 300 seconds or 5 minutes.

PASSIVE INFRARED SENSOR (PIR)

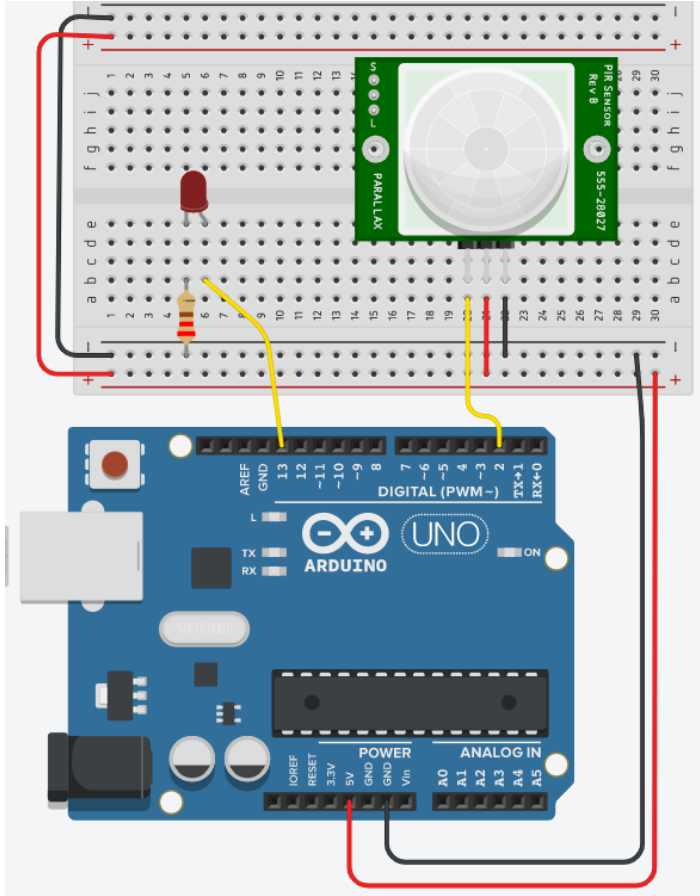
Example

- Turn ON the LED for a short time when movement is detected.
- Otherwise, turn it OFF.



PASSIVE INFRARED SENSOR (PIR)

```
int sensorState = 0;
void setup()
{
  pinMode(2, INPUT);
  pinMode(13, OUTPUT);
  Serial.begin(9600);
}
void loop()
{
  sensorState = digitalRead(2);
if (sensorState == HIGH) {
    digitalWrite(13, HIGH);
    Serial.println("Sensor activated!");
  } else {
    digitalWrite(13, LOW);
  }
  delay(10);}
```

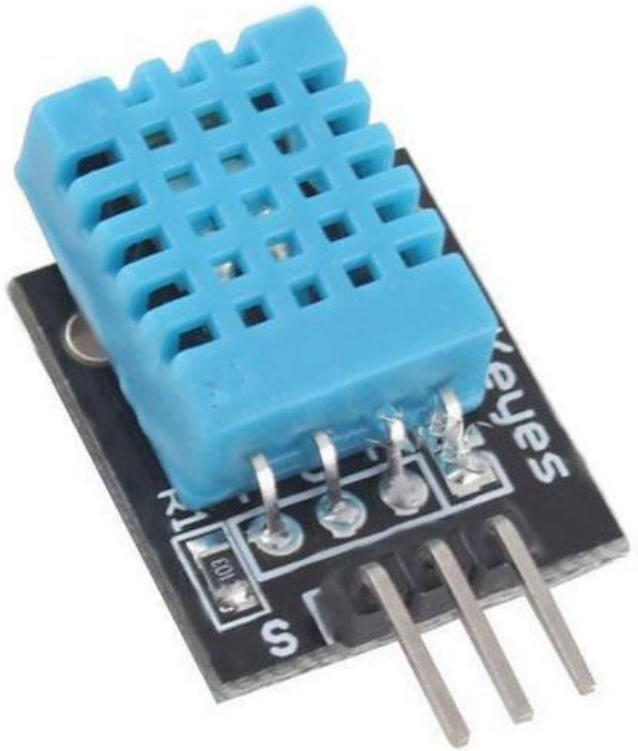


HUMIDITY AND TEMPERATURE SENSOR

DHT11

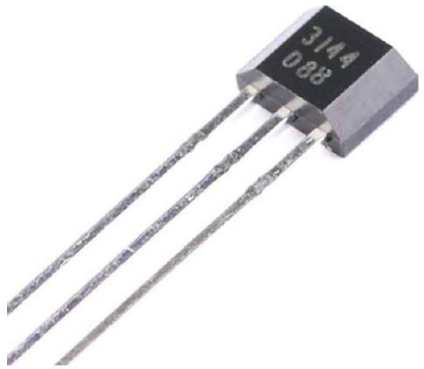
- The DHT11 sensor measures
  - temperatures between 0°C and 50°C
  - relative humidity between 20% and 90%
- Measurements frequency is 1 Hz
- Accuracy:
  - ±2°C for temperature
  - ±5% for relative humidity.

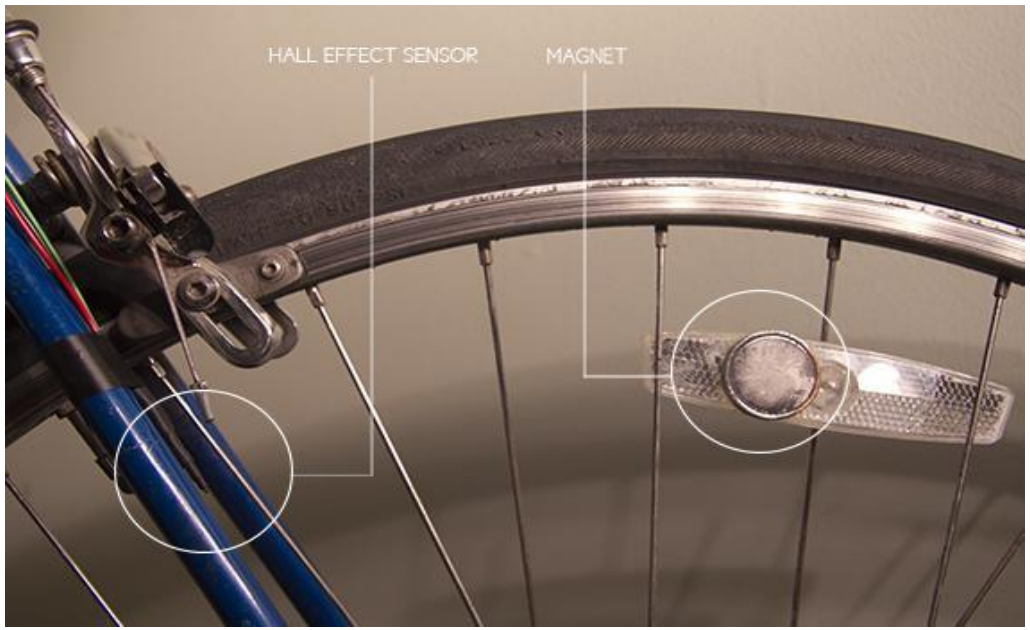
- Relative humidity = 0% -> the air is completely dry
- Relative humidity = 100% -> condensation occurs



HALL EFFECT SENSOR

- Hall effect sensors are activated by a magnet field
- Application example: Measuring bike speed





ACCELEROMETER AND GYROSCOPE

- An accelerometer measures an object's acceleration
- A gyroscope measures angular velocity
- Application example: Self-balancing bicycle